

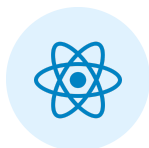
Quem aí é Fullstack?



slides.com/regissantos/inertiajs

Frontend

Os frameworks que montam a interface dentro do navegador.



ReactJS

O mais usado do mercado. Nasceu no Facebook.



VueJS

Leve e progressivo. Dá pra adotar aos poucos.



Svelte

Compila para JavaScript puro. Sem runtime pesado.



AngularJS

Completo e opinativo. Nasceu no Google.

Todos resolvem o mesmo problema: desenhar a tela no navegador.

Backend

As linguagens e frameworks que rodam no servidor.



Python

 Django

 FastAPI



Ruby

 Ruby on Rails



PHP

 Laravel



Elixir

 Phoenix

Todos fazem a mesma coisa: guardam os dados e aplicam as regras.

Backend, Frontend – e duas rotas

O jeito clássico: o Django vira API, o Vue consome. A mesma tela ganha duas rotas.



Duas rotas para a mesma tela — e dois sistemas de rotas para manter.

Web e app mobile

Às vezes a API precisa mesmo existir: quando ela tem mais de um consumidor.



Com mais de um cliente de verdade, a API deixa de ser custo e vira necessidade.



Regis Santos

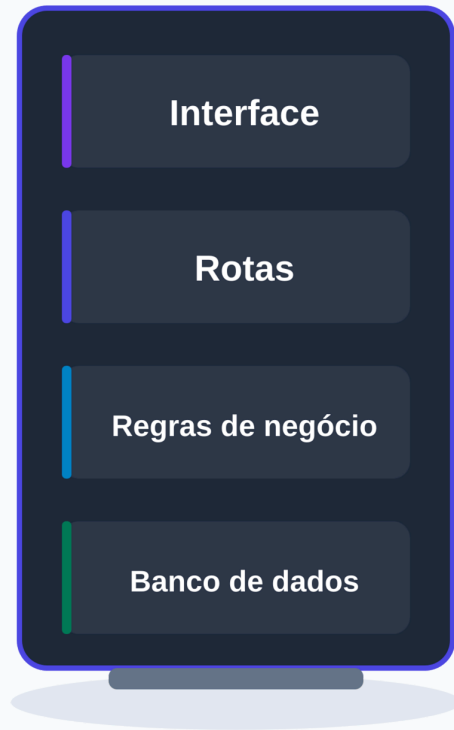


Regis do Python

Dev Python Senior + Django



django



Monolito

Freela

ou empresas que começaram com monolito

INERTIA.JS

O Monolito Moderno

Construindo SPAs com Django e Vue — sem precisar de uma API REST

Uma introdução para quem não conhece nenhuma das três



Django



Inertia



Vue



Django, em 30 segundos

Um framework web em Python com filosofia “baterias inclusas”.



ORM

Fala com o banco em Python, sem escrever SQL na mão



Autenticação

Login, sessão e permissões já vêm prontos



Roteamento

URLs e views organizadas no servidor



Renderiza HTML

Monta as páginas no servidor — guarde isso!

Guarde esta ideia: o Django pensa em HTML, no servidor.



Vue, em 30 segundos

Um framework JavaScript para construir interfaces no navegador.



Componentes reativos

- A tela é montada em peças reutilizáveis.
- Quando o dado muda, a tela atualiza sozinha.
- Sem recarregar a página a cada clique.



A sensação de aplicativo

- Navegação fluida, sem telas piscando.
- É o que dá o ar “moderno” à interface.
- React e Angular resolvem o mesmo problema.

O Vue pensa em componentes, no navegador.



Um exemplo do dia a dia

Uma lista de filmes reativa — o frontend é só pra você e sua equipe

 Minha Lista de Filmes

 tarantino

filtrando em tempo real...

Pulp Fiction (1994)

★★★★★

Assistido

Kill Bill (2003)

★★★★☆

Assistindo

Django Livre (2012)

☆☆☆☆☆

Quero Ver



Reativo

Digitou na busca, a lista filtra na hora — sem recarregar a página.



Dados em tempo real

Marcou como “assistido”? O status e a nota atualizam instantaneamente.



Consumo próprio

Esse frontend é da sua equipe. Ninguém de fora consome esses dados.

Como se faz hoje (e por que dói)



Caminho 1

API REST separada

O Django vira uma API que só fala JSON. O Vue vira um projeto à parte que consome essa API.

- Autenticação duplicada (sessão + JWT)
- Precisa configurar CORS
- Serializers + tipos pra sincronizar
- Dois sistemas de rotas
- Dois projetos, dois deploys



Caminho 2

Vue como “ilhas” no template

Pedaços de Vue colados dentro do HTML do Django, como widgets isolados.

- Cada navegação recarrega a página inteira
- Perde a fluidez de uma SPA
- Experiência de usuário travada
- “Voltamos para 2010”

O custo escondido

Você construiu uma API pública... que não é pública.



2

projetos para manter

2

sistemas de autenticação

0

clientes externos de fato

Você pagou o preço de uma arquitetura desacoplada — sem precisar do desacoplamento.

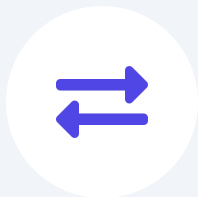
Entra o Inertia.js

“The Modern Monolith” — o monolito moderno



A sacada

E se o backend pudesse entregar componentes Vue em vez de HTML — passando os dados direto pro componente, sem virar uma API?



Uma camada fina, não um framework gigante

O Inertia substitui apenas o sistema de templates do Django. Onde antes ele renderizava um arquivo HTML, agora renderiza um componente Vue e injeta os dados. Todo o resto do Django — rotas, sessão, permissões, middleware, admin — continua exatamente igual.

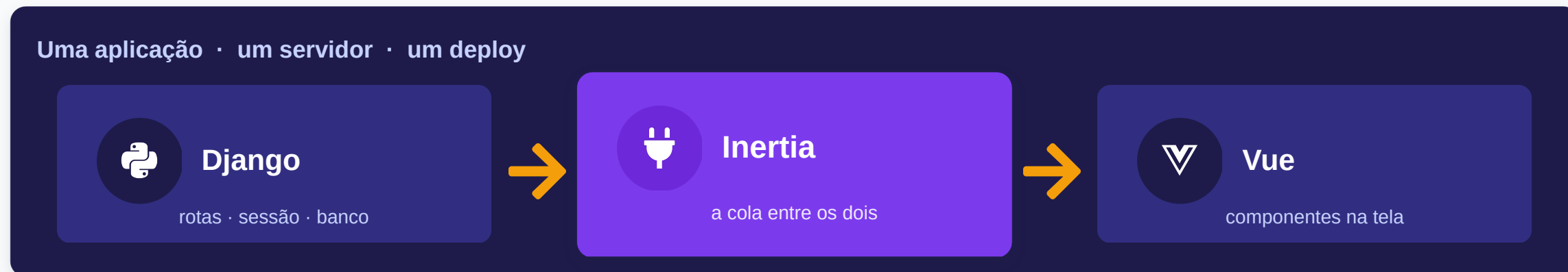
Onde o Inertia entra

A mesma aplicação — antes e depois do Inertia

ANTES — dois mundos separados



DEPOIS — um app só, o Inertia no meio



A ideia central

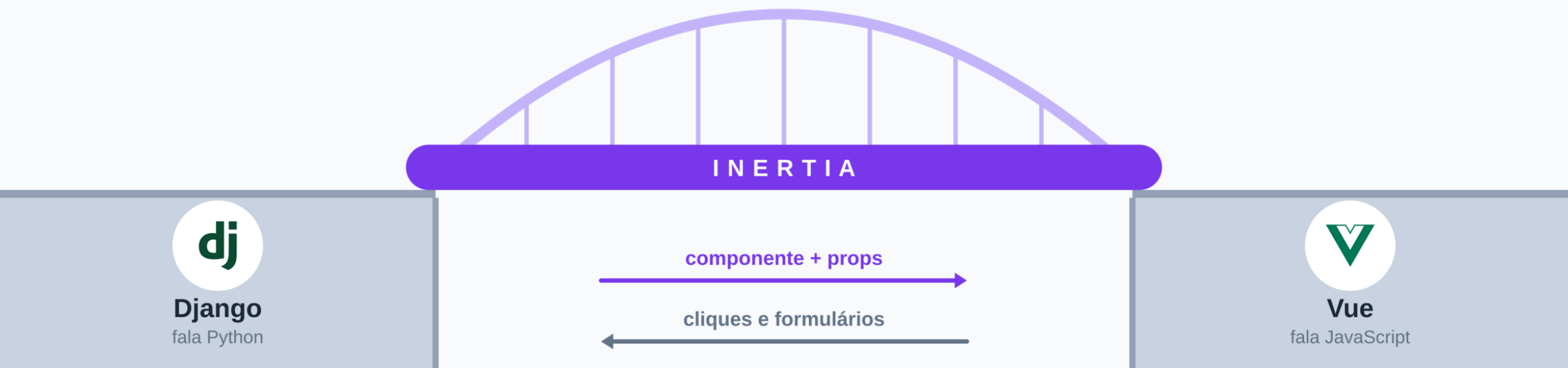
O Inertia é o tradutor simultâneo entre dois mundos



Nenhum dos dois precisa aprender a língua do outro.

O Inertia é a ponte

Um lado fala Python, o outro fala JavaScript. A ponte faz a travessia.



Sem API no meio: o que atravessa a ponte são componentes e props.

Como funciona uma requisição



1 — Primeira visita

O Django manda uma página HTML completa, com os dados já embutidos. Carrega rápido e funciona para SEO.



2 — Navegações seguintes

O Inertia intercepta o clique e pede só um JSON enxuto (cabeçalho X-Inertia). O Vue troca apenas o que mudou — sem recarregar.



3 — Envio de formulário

O Django processa, salva e faz um redirect — como sempre fez. O Inertia entende e busca a próxima página sozinho.



O código: a view no Django

```
# from django.shortcuts import render
from inertia import render

def index(request):
    movies = Movie.objects.all()
    return render(request, "Movies/Index", props={
        "movies": list(movies.values()),
    })
```

A única mudança: `render(request, "Movies/Index", props=...)`

O 2º argumento é um componente Vue; o 3º são os dados que vão direto pra ele. Sem serializer, sem JsonResponse, sem montar endpoint. É Python e Django puro.



O código: o componente Vue

```
<script setup>
// 'movies' chega direto da view Django
const props = defineProps(["movies"])
</script>

<template>
  <div v-for="m in movies">{{ m.title }}</div>
</template>
```

Sem Inertia, aqui teria:

- fetch / axios escrito à mão
- tratamento de loading e erro
- um store do Pinia pro estado

Com Inertia:

- os dados já estão lá, como props
- nenhuma chamada HTTP manual
- nenhum gerenciamento de estado

O que você ganha



Sem duplicação

Uma fonte de verdade no Django.
Nada de sincronizar serializer e tipo.



Auth no Django

Sessão, permissões e middleware
nativos. Sem token no navegador.



Um sistema de rotas

As URLs vivem no Django. Vue
Router nem existe.



SPA de verdade

Navegação fluida, sem reload, com
barra de progresso.



Um projeto, um deploy

Sem CORS, sem dois repositórios,
sem versão de API.



Menos código

A melhor complexidade é a que você
nunca escreveu.

Instalação

O adaptador oficial do Inertia para Django.

REPOSITÓRIO

github.com/inertiajs/inertia-django

```
$ uv add inertia-django
```

Configuração

Editar o settings.py

```
INSTALLED_APPS = [  
    # django apps,  
    'inertia',  
    # your project's apps,  
]
```

```
MIDDLEWARE = [  
    # django middleware,  
    'inertia.middleware.InertiaMiddleware',  
    # your project's middleware,  
]
```

CSRF

```
createInertiaApp({  
    // ...  
    http: {  
        xsrfCookieName: 'csrftoken',  
        xsrfHeaderName: 'X-CSRFToken',  
    },  
})
```


Com Inertia vs. sem Inertia

	Com Inertia	Sem Inertia (DRF + Vue)
API REST	Não precisa	Obrigatória
Serializers	Não precisa	Obrigatórios
CORS	Não precisa	Configurar e manter
Autenticação	Sessão do Django	Duplicada (sessão + JWT)
Vue Router	Não existe	Obrigatório
Estado global (Pinia)	Não precisa	Necessário
Projetos / deploys	Um	Dois

Cada “Não precisa” é uma coisa a menos para manter.

Quando o Inertia NÃO serve

Honestidade técnica: toda ferramenta tem limite.



Se você usa FastAPI

É incompatibilidade de design.

- O FastAPI nasce como API pura
- Não tem template para substituir
- Não tem roteamento de páginas
- Caminho certo: Vue separado + Pinia



Se a API é de verdade

Quando há outros consumidores além do seu frontend.

- App mobile consumindo os dados
- Terceiros / parceiros via API
- Vários clientes diferentes
- Aí a API desacoplada faz sentido



A pergunta que resume tudo

“Quem vai consumir isso, além do meu próprio frontend?”

Se a resposta for “ninguém”... talvez você não precise de uma API. Talvez você precise do Inertia.

Para levar pra casa



Não é bala de prata, e não substitui toda API.



Resolve um problema específico e muito comum.



Interface moderna sem abrir mão do monolito.

Se você é uma equipe Django construindo para os seus próprios usuários, o Inertia provavelmente vai apagar uma camada inteira de complexidade.

Obrigado! • Perguntas?

O código está no GitHub

E os slides também



github.com/rg3915/django-inertia-vuejs

Obrigado! • Perguntas?